

Pipelines de CI/CD: Concepto de la Industria e Implementación en SICAD

Guía pedagógica y técnica que explica el funcionamiento, estructura e importancia de las pipelines de Integración Continua (CI), Entrega Continua (CD) y Gestión de Releases en el mundo profesional y cómo estas prácticas automatizadas protegen y despliegan la aplicación SICAD.

Elaborado por: Joel Acevedo (Estudiante & Desarrollador)

Proyecto: SICAD (Sistema de Información de Control y Administración Docente)

Destino de Despliegue: Microsoft Azure App Service (Docker Containers)

Fecha: Julio 2026

Estatus del Documento: Entregable de Pruebas y Despliegue

1. Las Pipelines de CI/CD en el Mundo Laboral

En el desarrollo de software profesional, una pipeline (o tubería de automatización) representa el conjunto de pasos automatizados que permiten transformar el código fuente escrito por los desarrolladores en una aplicación funcional ejecutándose de forma segura en producción. La implementación de pipelines forma parte de la filosofía DevOps, cuyo fin es estrechar la brecha entre el desarrollo de software y la administración de sistemas.

Los dos pilares de este proceso son:

- Integración Continua (CI): Es la práctica de compilar y probar el código de forma automática cada vez que un desarrollador sube cambios al repositorio común. El objetivo primordial de la CI es detectar errores de lógica, de estilo o de dependencias de manera inmediata, evitando el 'infierno de la integración' que ocurre cuando se intenta unir el trabajo de múltiples personas tras semanas de desarrollo aislado.
- Entrega y Despliegue Continuo (CD): Va un paso más allá de la CI. Consiste en automatizar la preparación del software para su puesta en producción. En la Entrega Continua (Continuous Delivery), cada cambio exitoso en CI se empaqueta y queda listo para desplegarse a producción, requiriendo una autorización manual. En el Despliegue Continuo (Continuous Deployment), el proceso es 100% automático: si pasa las pruebas en la pipeline, se publica directamente en el servidor sin intervención humana.

Beneficio Industrial: En el mundo profesional, las pipelines eliminan las tareas manuales repetitivas y propensas a errores, garantizando que el software desplegado sea siempre funcional, rastreable y seguro.

Fases Típicas en una Pipeline Profesional:

1. Compilación (Build): Descarga del código, instalación limpia de dependencias y empaquetado del software (por ejemplo, compilando JavaScript o empaquetando todo en contenedores Docker).
2. Calidad Estática (Linting): Análisis automatizado del código sin ejecutarlo para asegurar que cumple con las reglas de estilo (ESLint) y no posee vulnerabilidades críticas conocidas.
3. Pruebas Automatizadas (Testing): Ejecución de suites de pruebas unitarias (prueban funciones individuales) e integración (prueban el flujo entre módulos) usando herramientas de testing (como Jest o PyTest).
4. Publicación de Artefactos: Envío de la versión construida (los binarios, archivos compilados o imágenes de contenedor Docker) a un registro central seguro (como GitHub Packages, Docker Hub o Azure Container Registry).
5. Despliegue (Deployment): Instalación automática de los artefactos en los servidores finales (como AWS, Azure o Kubernetes), a menudo dividida en fases de pruebas (Staging/QA) y producción final.

2. Estructura y Funcionamiento de las Pipelines en SICAD

El proyecto SICAD incorpora una arquitectura de CI/CD profesional y moderna, configurada mediante GitHub Actions. Se han estructurado tres flujos independientes (workflows) alojados en el directorio `.github/workflows/`. Estos flujos controlan de forma segura el ciclo de vida del software, desde que se propone un cambio hasta que se publica oficialmente en la nube.

Resumen de las Pipelines de SICAD:

Pipeline (Archivo)	Disparador (Trigger)	Tareas Clave (Jobs)	Destino / Plataforma
CI - Build & Test <code>ci.yml</code>	Push o Pull Request a dev, qa y main	• Lint del frontend (Next.js) • Build del frontend • Generación Prisma Client • Pruebas Jest backend	GitHub Runners (Ubuntu Linux)
CD - Deploy a Azure <code>deploy.yml</code>	Push a main o lanzamiento manual	• Construcción Docker (Frontend y Backend) • Login y Publicación en GitHub Registry • Despliegue en Azure	Azure App Service (sicad-backend / sicad-frontend)
Release <code>release.yml</code>	Push de tags con formato de versión v*	• Construcción local Docker para verificar empaque • Publicación de release oficial • Auto-generación de notas	GitHub Releases

2.1 Integración Continua (CI): `ci.yml`

Este workflow actúa como el primer filtro de calidad. Se ejecuta ante cualquier propuesta de cambio (pull request o push) en las ramas activas de desarrollo. Esto asegura que nadie pueda subir cambios que rompan la compilación o las pruebas.

Trabajo del Backend (Lint + Pruebas):

- Descarga el código y configura Node.js 20 utilizando caché de dependencias.
- Ejecuta `npm ci` para realizar una instalación exacta y limpia de las librerías.
- Genera el cliente de base de datos con `npx prisma generate` (crucial para TypeScript y la interacción con la base de datos).
- Corre las pruebas unitarias y de integración mediante Jest (`npm test`) inyectando secretos ficticios de entorno para la suite de pruebas.

Trabajo del Frontend (Build & Lint):

- Configura el entorno y descarga las dependencias usando la misma estrategia de caché limpia.
- Ejecuta `npm run lint` (ESLint) para asegurar que el código cumple con el estándar estético del proyecto.
- Corre `npm run build`, compilando la aplicación Next.js y garantizando que no haya errores sintácticos o de tipado en el frontend.

2.2 Despliegue Continuo (CD) a Azure: deploy.yml

Una vez que el código es validado en la rama de pruebas (QA) y se mezcla en la rama de producción (main), se dispara automáticamente este flujo. Su propósito es empaquetar la aplicación en tecnologías modernas de contenedores (Docker) y actualizar las aplicaciones en la nube de Azure de manera transparente.

Flujo de Ejecución en Dos Trabajos Secuenciales:

1. Construcción y Publicación (build-push): Inicia sesión de manera segura en el registro de contenedores de GitHub (ghcr.io). Construye las imágenes Docker para el backend (sicad-backend) y el frontend (sicad-frontend) usando sus respectivos Dockerfiles. Inyecta la variable pública de API (NEXT_PUBLIC_API_URL) en el frontend y sube los contenedores con las etiquetas de versión (hash git) y latest.
2. Despliegue a Azure (deploy): Espera que el paso de construcción termine con éxito. Utiliza credenciales seguras de Azure (AZURE_CREDENTIALS) para iniciar sesión en la consola corporativa de Azure y actualiza los servicios Azure App Service correspondientes, obligando a los servidores en la nube a descargar las nuevas imágenes Docker recién construidas.

Principio de Inmutabilidad: El uso de Docker en SICAD garantiza el principio de Inmutabilidad: el mismo contenedor que fue probado en el entorno de compilación es el que se despliega en producción en Azure, evitando problemas asociados a diferencias de configuraciones en servidores remotos.

2.3 Pipeline de Release: release.yml

Este pipeline se encarga de empaquetar y marcar hitos oficiales en el desarrollo. No se ejecuta con los commits ordinarios, sino que responde a los tags de Git que siguen el formato de versionado semántico (ejemplo: v1.2.0).

Fases de la Publicación de Versiones:

- Validación Local (build-images): Prepara el entorno Docker Buildx y compila localmente los Dockerfiles de frontend y backend etiquetándolos con el nombre exacto de la versión (ej. sicad-backend:v1.2.0). Esto asegura que el empaquetado del tag es correcto.
- Creación del Release (release): Utiliza la acción softprops/action-gh-release@v2 para crear una publicación formal en la pestaña de Releases de GitHub. Genera notas de la versión de manera automatizada basadas en los mensajes de commit, manteniendo a todo el equipo al tanto de las novedades introducidas en dicha versión.

Conclusión

Al integrar estas tres pipelines, el desarrollo de SICAD adopta los mejores estándares de la industria tecnológica. Esto proporciona al equipo un entorno seguro donde se puede iterar velozmente sin miedo a romper la aplicación de producción, garantizando la disponibilidad continua y la estabilidad del sistema para todos los usuarios.